Distributed-Memory Parallelisation of a Barnes–Hut *N*-Body Simulation

IT4130E Parallel and Distributed Programming

[member 1] *[member 2]* *[member 3]* *[member 4]*

[instructor name]

June 14, 2026

Outline

The N-body problem

- N masses attract each other by Newtonian gravity; advance their motion through time.
- Force on a body sums contributions of all others:

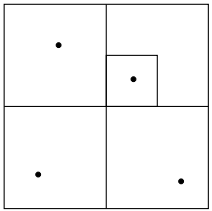
$$\vec{a}_i = G \sum_{j \neq i} m_j \frac{\vec{r}_j - \vec{r}_i}{(\|\vec{r}_j - \vec{r}_i\|^2 + \varepsilon^2)^{3/2}}$$

- Direct evaluation costs $\mathcal{O}(N^2)$ per step — too slow.
- **Goal:** a correct sequential method, a distributed-memory (MPI) parallel version, and a hybrid MPI + OpenMP version, with a measured performance study.

Barnes–Hut: $\mathcal{O}(N \log N)$

- Recursively divide the plane into a **quadtree**; each leaf holds ≤ 1 body.
- Each node stores the total mass and centre of mass below it.
- Force on a body: traverse from the root. If a node is far enough ($s/d < \theta$), treat the whole subtree as one mass; else descend.
- $\theta = 0.5$: about 1% force error, $\mathcal{O}(N \log N)$ work.
- The tree makes the work **irregular** — the key parallel challenge.

$s/d < \theta \Rightarrow$ approximate



Design choices (the rubric questions)

- **Level of parallelism:** *data* — same operation, different bodies.
- **Decomposition:** *data decomposition* (owner-computes per body) over a *recursive* (replicated) tree build $\Rightarrow$ hybrid, data-dominated.
- **Mapping:** *1-D block* — process r owns a contiguous block of $\approx N/P$ bodies (not 2-D $\sqrt{P} \times \sqrt{P}$: the domain is a 1-D list, not a grid).
- **Communication:** *blocking, collective*; broadcast once at start, then one *all-gather* of positions per step. Symmetric single-program multiple-data (SPMD), *no* master–slave; no explicit ring/hypercube topology.
- **Load balance:** equal block *size*; work per body is uneven (dense regions cost more).

Parallel algorithm (process r of P)

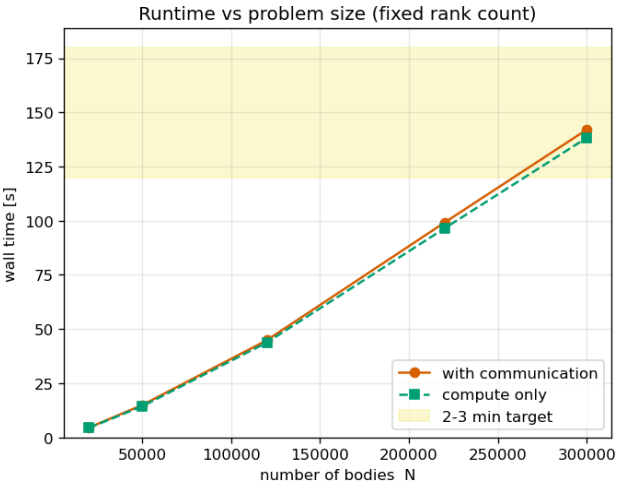
1: **(once)** broadcast initial positions, velocities, masses
2: $lo, hi \leftarrow$ block owned by process r ▷ 1-D block
3: **for** each time step **do**
4: build the full quadtree locally ▷ replicated
5: accumulate mass / centre of mass at each node
6: **for** $i = lo \ldots hi - 1$ **do** ▷ OpenMP parallel-for in hybrid
7: $\vec{a}_i \leftarrow$ traverse tree with θ
8: **end for**
9: $\vec{v}_i \mathrel{+}= \vec{a}_i \Delta t; \quad \vec{r}_i \mathrel{+}= \vec{v}_i \Delta t$
10: **all-gather** updated positions
11: **end for**

Force traversal communicates nothing (tree is complete on every process). Cost: build $\mathcal{O}(N)$ replicated + force $\mathcal{O}((N/P)\log N)$ + comm $\mathcal{O}(N)$.

Correctness

- Parallel and hybrid outputs match the sequential reference **bit for bit**, for every process and thread count tested.
  - Each body is integrated by identical arithmetic; the all-gather moves data without changing it.
- Total energy conserved to within a fraction of a percent over the run $\Rightarrow$ the model is physically stable.
- The cluster stays bound and near equilibrium (title figure).

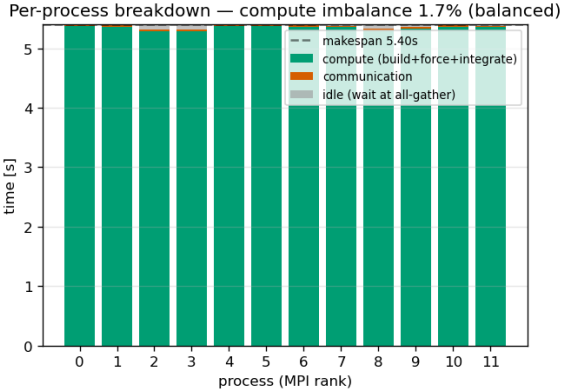
Running time vs problem size (choosing N)



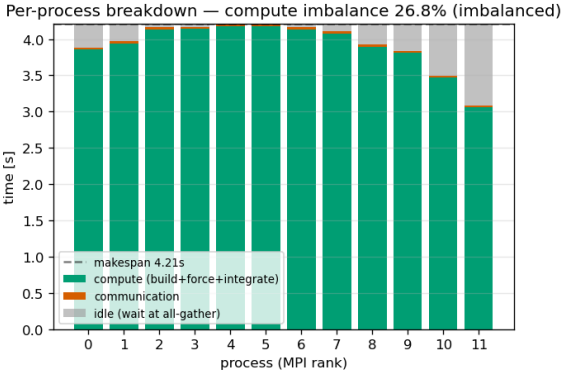
- Twelve processes (= cores), vary N.
- Time grows $\sim N \log N$; communication is a small fraction.
- Pick the N landing in the 2–3 minute band as the operating point for the load-balance study.

Granularity & load balance (25% idle rule)

Balanced (generated order)

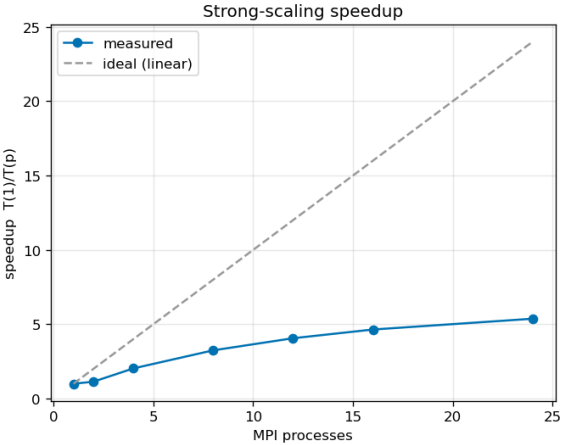


Imbalanced (sorted by position)



Equal block *size* = equal *work* only when ordering is unrelated to position. Fix: cyclic distribution / dynamic thread schedule.

Speed-up



- Fixed size, double the processes $1, 2, 4, \ldots, 2X$.
- Speed-up saturates: the **replicated tree build** is not divided $\Rightarrow$ Amdahl ceiling.
- Communication is the second, smaller cost (gap between runtime curves).

Conclusion

- Data-parallel Barnes–Hut: force work divides cleanly and communicates nothing; result is identical to sequential.
- Clear speed-up, with an efficiency ceiling *explained* by the replicated build (Amdahl) and the per-step all-gather.
- Load balance follows directly from the mapping and the input ordering.
- **Future work:** distribute the tree (remove the sequential build), cyclic / cost-aware mapping, extend to 3-D (octree).

Thank you — questions?